

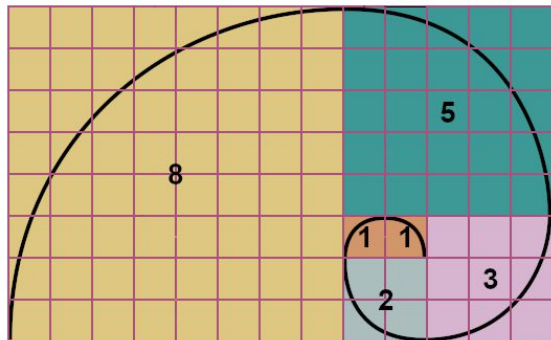
Programação Funcional

Prática a04 - Recursividade

Ficha PL1 — Parte 4

Capítulo 5 — Recursion, pp. 38–43

O que é recursividade?

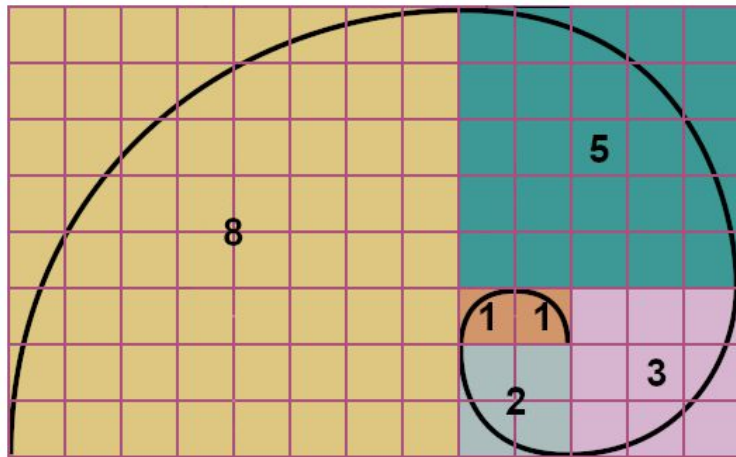


- $F_1 = 1$ e $F_2 = 1$
- $F_n = F_{n-1} + F_{n-2}$ (para $n \geq 3$)

- Uma função recursiva é uma função que se chama a si própria.
- Resolver um problema usando a solução de uma versão mais pequena do mesmo problema.

Mas se uma função está sempre a chamar-se a si própria, como é que isto alguma vez termina?

As duas peças fundamentais



$F_1 = 1$ e $F_2 = 1 \rightarrow$ **Caso base** (Um caso que sabemos resolver diretamente)

$F_n = F_{n-1} + F_{n-2}$ (para $n \geq 3$) $\rightarrow$ **Caso recursivo** (Resolve o problema recorrendo a uma versão mais pequena do mesmo problema.)

Exploração inicial em GHCi

FATORIAL

- $\text{fatorial } 0 = 1 \rightarrow$ Caso base
- $\text{fatorial } n = n * \text{fatorial } (n - 1) \rightarrow$ Caso recursivo

```
rec.hs U X
aulas > a04 > rec.hs
1  fatorial :: Int -> Int
2  fatorial 0 = 1
3  fatorial n = n * fatorial (n - 1)
4
```

Qual é o problema que está a ficar mais pequeno? n passa para $n - 1$

O que acontece em fatorial 4?

fatorial 4

→ $4 * \text{fatorial } 3$

→ $4 * 3 * \text{fatorial } 2$

→ $4 * 3 * 2 * \text{fatorial } 1$

→ $4 * 3 * 2 * 1 * \text{fatorial } 0$

→ $4 * 3 * 2 * 1 * 1$

→ 24

FATORIAL

- $\text{fatorial } 0 = 1 \rightarrow$ Caso base
- $\text{fatorial } n = n * \text{fatorial } (n - 1) \rightarrow$ Caso recursivo

Antes de escrever uma função recursiva...

Pergunta sempre:

- Qual é o caso mais simples?
- Qual é a resposta nesse caso?
- Como posso tornar o problema mais pequeno?
- Como uso a solução do problema mais pequeno para construir a resposta?

Recursão sobre listas

Relembrar:

- `[]` → lista vazia
- `(x:xs)` → `x`: head; `xs`: tail

Como calculamos a soma de `[1,2,3,4]`?

`soma [1,2,3,4] = 1+soma [2,3,4] = 1+2+soma [3,4]...`

`soma [] = 0` → Caso base

`soma (x:xs) = x + soma xs` → Caso recursivo

```
rec.hs U X
aulas > a04 > rec.hs
1  soma :: [Int] -> Int
2  soma [] = 0
3  soma (x:xs) = x + soma xs
4
```

Recursão sobre listas

Como calculamos o comprimento de uma lista?

$\text{comprimento } [] = 0 \rightarrow$ Caso base, o comprimento de uma lista vazia é 0

$\text{comprimento } (_ : xs) = 1 + \text{comprimento } xs \rightarrow$ Caso recursivo, 1 + o comprimento da tail da lista

```
» rec.hs U X
aulas > a04 > » rec.hs
1  comprimento :: [a] -> Int
2  comprimento [] = 0
3  comprimento (_:xs) = 1 + comprimento xs
4
```

Exercícios

No mesmo ficheiro `rec.hs`, implementar e testar as seguintes funções:

- `produto` Função que recebe uma lista e devolve o produto dos elementos.
- `duplica` Função que recebe uma lista de números e devolve uma nova lista com cada elemento duplicado (*2).
- `contaPositivos` Função que recebe uma lista de números e devolve a quantidade de elementos positivos.
- `todosPositivos` Função que recebe uma lista de números e devolve True se todos os elementos forem positivos e False caso contrário.
- `existeZero` Função que recebe uma lista de números e devolve True se existir pelo menos um zero e False caso contrário.

```
duplica :: [Int] -> [Int]
```

```
-- duplica recebe uma lista de Int e devolve uma lista de Int.
```

```
duplica [] = []
```

```
-- Caso base: duplicar os elementos de uma lista vazia dá uma lista vazia.
```

```
duplica (x:xs) = (2 * x) : duplica xs
```

```
-- Caso recursivo:
```

```
-- x é o primeiro elemento da lista e xs é o resto da lista.
```

```
-- Multiplicamos x por 2.
```

```
-- Depois usamos : para colocar esse valor à frente da lista resultante de duplica xs.
```

```
-- A chamada duplica xs trata recursivamente o resto da lista.
```

```
contaPositivos :: [Int] -> Int
```

```
-- contaPositivos recebe uma lista de Int e devolve um Int,  
-- correspondente ao número de elementos positivos da lista.
```

```
contaPositivos [] = 0
```

```
-- Caso base: numa lista vazia existem 0 elementos positivos.
```

```
contaPositivos (x:xs)
```

```
  | x > 0      = 1 + contaPositivos xs
```

```
-- Se o primeiro elemento x for positivo, contamos 1  
-- e continuamos a contar os positivos no resto da lista xs.
```

```
  | otherwise = contaPositivos xs
```

```
-- Caso contrário, não contamos x  
-- e continuamos apenas com o resto da lista.
```

```
existeZero :: [Int] -> Bool
```

```
-- existeZero recebe uma lista de Int e devolve um Bool:
```

```
-- True se existir pelo menos um zero e False caso contrário.
```

```
existeZero [] = False
```

```
-- Caso base: numa lista vazia não existe nenhum zero.
```

```
existeZero (x:xs)
```

```
  | x == 0    = True
```

```
-- Se o primeiro elemento x for 0, já encontramos um zero,
```

```
-- por isso podemos devolver imediatamente True.
```

```
  | otherwise = existeZero xs
```

```
-- Caso contrário, procuramos um zero recursivamente
```

```
-- no resto da lista xs.
```

Ficha PL1 - Parte 4

no git <https://github.com/v2denny/pf2627>
pasta >aulas>a04>FichaPL1.pdf

Desafio

Implementa a função: `replica :: Int -> a -> [a]`, sem usar `replicate`.

A função deve receber um número inteiro `n` e um valor, e devolver uma lista com esse valor repetido `n` vezes.

Exemplos para testar:

```
</> haskell
```

```
replica 4 'a'    -- "aaaa"  
replica 3 7      -- [7,7,7]  
replica 0 5      -- []
```